

GO PROGRAMMING MASTER COURSE

Week 8, Day 1

Pointers, memory pictures, and a
StudentRepository — HTTP stops owning the
map.

Umesh Indrajith · BSc Eng (Hons), University of Moratuwa
Senior Software Engineer / Lecturer

SINCE
LAST
WEEK

Students live in the handler. That was temporary.

- POST / GET / GET:id still work from an in-memory map.
- Cold-call: what is a pointer receiver? Why did handlers use `*StudentHandler`?
- Today we name that idea properly — then extract storage.
- Still no database. Repository shape matches this repo; map underneath.

TODAY

If Day 1 of week 8 works, you can

- Read and write through & and *; explain a nil pointer panic.
- Say why maps/slices already share backing data.
- Sketch stack vs heap / GC at picture level (not assembly).
- Move the student map into internal/repository.
- Keep handlers talking HTTP only — they call the repo.

SOFTWARE
ENGINEERING

HTTP must not own storage forever

A repository
hides where
data lives.

Handlers stay
thin: bind
JSON, call repo,
map results to
status codes.

- Same shape as `backend/internal/repository/student_repository.go` — without `pgx` yet.
- Week 14 swaps the map for SQL; handlers should barely change.
- If Create SQL (or map writes) sit inside a Gin handler, the layering is lying.

POINTERS

& address · * value at address

```
x := 10  
p := &x      // p is *int  
fmt.Println(*p) // 10  
*p = 20  
fmt.Println(x)  // 20
```

- A pointer holds a memory address, not the value itself.
- Methods that must share or mutate one instance use pointer receivers.

NIL
POINTERS

Zero value of a pointer is nil

```
var h *handlers.StudentHandler  
h.List(c) // panic: nil
```

- Nil means “no address yet” — not an empty struct.
- Always construct with `NewStudentHandler(repo)` (or `&StudentHandler{...}`) before use.
- Lab starts with a deliberate crash, then a one-line fix.

POINTER
RECEIVERS

Why handlers use *StudentHandler

```
type StudentHandler struct {  
    repo *repository.StudentRe  
}  
func (h *StudentHandler) Create
```

- One handler instance is shared across requests — it holds the repo (later: deps).
- Value receiver would copy the struct; pointer keeps the same h.
- Course default for HTTP handlers: pointer receivers.

VALUE VS
REFERENCE

Structs copy · maps and slices share

Struct / int

Passed by value — callee gets a copy unless you pass a pointer.

Map / slice header

Header is copied; backing array/buckets are shared. Mutations show up.

String

Immutable bytes; header copied. Changing needs a new string.

Pointer

Explicit shared memory. Check for nil before dereference.

STACK VS HEAP ·
PICTURE

Where values live (enough for today)

- **Stack** — short-lived locals; cheap; gone when the function returns.
- **Heap** — longer-lived / shared; GC cleans unreachable objects.
- Returning `&local` or storing a pointer in a long-lived struct → often heap.
- You do not manage `free()` — the runtime does.

ESCAPE
ANALYSIS ·
ONE SLIDE

Does this value outlive the function?

```
func NewStudentRepository() *StudentRepository {  
    return &StudentRepository{  
    }  
}
```

- Compiler decides stack vs heap.
go build -gcflags="-m" can show escapes — optional demo.
- Rule of thumb: if something else keeps a pointer to it, it escapes.

**GARBAGE
COLLECTION**

Unreachable = reclaimable

- When nothing points at an object, GC may free it later.
- Leaks in Go are usually *still-reachable* junk (maps that grow forever, forgotten goroutines — later weeks).
- Today's map store lives until process exit — that is fine for week 8.

MAKE VS
NEW ·
LIGHT

Only if it comes up

```
m := make(map[int]models.Student) // read  
p := new(StudentRepository)       // *T z
```

- Prefer make for maps/slices/channels; constructors for structs.
- Course style: NewStudentRepository() that makes the map.

**SMS
MILESTONE**

StudentRepository owns the map

```
backend/internal/repository/student.go  
Create / GetAll / GetByID (/ u
```

```
handlers.StudentHandler { repo *  
// handlers call repo - no map f
```

Routes and status codes stay the same as week 7.

No JWT. No Postgres. Restart still clears memory.

LIVE
DEMONIL
CRASH

Break it on purpose

```
var h *handlers.StudentHandler
r.GET("/api/students", h.List)
// first request → panic: runtime
```

Show the stack trace. Name the bug: nil receiver.

Fix: `h := handlers.NewStudentHandler(repo)` before wiring routes.

LIVE
DEMO

REPOSITORY

Extract the map

```
type StudentRepository struct {  
    mu      sync.Mutex  
    students map[int]models.Student  
    nextID   int  
}  
  
func NewStudentRepository() *StudentRepository {  
    return &StudentRepository{students: map[int]models.Student  
}
```

LIVE
DEMO

HANDLER

Thin HTTP over the repo

```
func NewStudentHandler(repo *repository.StudentRepository) StudentHandler {
    return &StudentHandler{repo: repo}
}

func (h *StudentHandler) GetByID(c *gin.Context) {
    id, err := strconv.Atoi(c.Param("id"))
    // ...
    s, ok := h.repo.GetByID(id)
    if !ok { c.JSON(404, gin.H{"error": "student not found"}) }
    c.JSON(200, s)
}
```

WIRING

What main looks like after

```
repo := repository.NewStudentRepository()
students := handlers.NewStudentHandlers(repo)
r.POST("/api/students", students.CreateStudent)
r.GET("/api/students", students.GetStudents)
r.GET("/api/students/:id", students.GetStudent)
```

Construct repo first, inject into handler — dependency injection without a framework.

Later weeks inject DB pool the same way.

**NOT
FOUND**

Repo returns data · handler picks status

- Repo: (Student, bool) or (Student, error) — pick one style and stick to it.
- Handler maps missing → 404, bad id → 400, success → 200.
- Do not import gin inside repository.

Done when all of this is true

- Nil-pointer crash demonstrated (or narrated) and fixed
- `internal/repository/student_repository.go` owns the map
- Handler holds `*StudentRepository`; no map fields on handler
- Postman create + list + get still pass; health green
- PR started describing the move

HOMEWORK

PR the extract + draw memory

- Handler → repository for create/list/get (and update/delete if you have them).
- PR description: notebook photo or ASCII of stack/heap for &x / nil crash.
- Still no JWT / Postgres.
- Stretch: repo returns error; handler maps ErrNotFound → 404 (week 9 preview).

**NEXT
SESSION**

Week 9 — errors, validation, slog, JWT preview

- `fmt.Errorf` with `%w`, `errors.Is` / `As`.
- Consistent `{"error": "..."};` structured logs.
- Bring a clean handler → repository split.

WORDS TO
KEEP

Say these correctly next week

Pointer

Address of a value; nil means none.

Receiver

Method's target; pointer shares one instance.

Repository

Storage API; hides map or SQL.

Escape

Value outlives the function
→ often heap.

Where the new files live

```
backend/  
  cmd/server/main.go           # wire repo → handler  
  internal/models/student.go  
  internal/repository/  
    student_repository.go      # map + mutex + CRUD  
  internal/handlers/  
    student.go                 # HTTP only; holds *St
```

What this week is not

- Not Postgres — repository still wraps a map.
- Not a deep OS memory course — pictures beat dumps.
- Not interface-heavy DI — concrete `*StudentRepository` is fine.
- Races under load still week 12.

SAFETY

Still true while refactoring

- Commit working week-7 routes before the extract.
- Do not “fix” by deleting go.mod.
- gofmt; no secrets; port :8080.

Questions